DOCUMENTATION TECHNIQUE

Automatisation SQL par CronJob Kubernetes

Architecture technique et lancement des scripts SQL

Image : ahceneaiti/bash_cj:1.1 (busybox 1.36-musl)
Base : MySQL 8.4, base « ecommerce » (docker compose)
Cluster : kind, namespace sql-updates
Dépôt : bash_sql, branche k8s
Document du 9 novembre 2026

Sommaire

1. Objectif et périmètre 4

1.1 Contexte 4

1.2 Objectifs 4

1.3 Avant et après 4

1.4 Périmètre 4

2. Architecture technique 5

2.1 Vue d’ensemble 5

2.2 Composants 5

2.3 Déroulement d’une exécution 5

2.4 Réseau 6

2.5 Choix de conception 7

3. L’image ahceneaiti/bash_cj:1.1 7

3.1 Dockerfile.cronjob 7

3.2 Explications 8

3.3 Pourquoi mariadb-connector-c 8

3.4 Contenu et taille 8

3.5 Construction, publication et versions 9

4. Manifestes Kubernetes 9

4.1 ConfigMap 9

4.2 Secret 10

4.3 CronJob 10

4.4 Job manuel (job-manuel.yaml) 11

4.5 Script run-job.sh 11

5. Le script select_update.sh 12

5.1 Principe 12

5.2 Arguments et code retour 12

5.3 Variables d’environnement 12

5.4 Traitement d’un job 13

5.5 Convention de nommage des fichiers 13

5.6 Journalisation et rétention 13

5.7 Attente de la base et gestion des erreurs 13

6. Les jobs SQL fournis 14

6.1 Job « commandes » 14

6.2 Job « livraisons » 14

6.3 Job « produits » 15

6.4 Exemple de sortie . . . 15
6.5 Ajouter un job . . . 15
7. Modes de lancement . . . 16
7.1 Lancement planifié . . . 16
7.2 Lancement à la demande avec run-job.sh . . . 16
7.3 Job manuel (manifeste) . . . 16
7.4 Un seul job planifié : args dans cronjob.yaml . . . 16
7.5 Comparatif . . . 17
8. Mise en route pas à pas . . . 17
9. Exploitation . . . 18
9.1 Commandes courantes . . . 18
9.2 Logs et rétention . . . 18
9.3 Publier une nouvelle version . . . 18
9.4 Modifier la configuration . . . 18
10. Dépannage . . . 19
11. Sécurité et limites . . . 19
11.1 Sécurité . . . 19
11.2 Limites connues . . . 19
11.3 Évolutions possibles . . . 20
Annexe A. Arborescence du dépôt . . . 20
Annexe B. Historique des commits liés au CronJob . . . 21
Annexe C. Script select_update.sh (intégral) . . . 22
Annexe D. Manifeste job-manuel.yaml . . . 24
Annexe E. Script k8s/run-job.sh . . . 25

1. Objectif et périmètre

Ce document décrit comment des mises à jour SQL récurrentes sont automatisées avec un **CronJob Kubernetes**. Un script bash générique exécute, à heure fixe, des requêtes *SELECT* puis *UPDATE* sur une base MySQL 8.4. Le document couvre l'architecture, l'image du conteneur, les manifestes, le fonctionnement du script, les jobs SQL fournis, les différents modes de lancement, la mise en route et l'exploitation.

1.1 Contexte

La base **ecommerce** contient quatre tables liées (*client*, *produit*, *commande*, *livraison*) ainsi qu'une table *historique* alimentée par des triggers *AFTER UPDATE* : chaque modification faite par un job y est enregistrée avec l'état de la ligne avant et après (colonnes JSON *valeurs_avant* et *valeurs_apres*). Le serveur MySQL tourne dans un conteneur géré par docker compose (*ecommerce-mysql*, port 3306 publié sur l'hôte).

1.2 Objectifs

- Exécuter les mises à jour automatiquement, toutes les heures pleines de 6 h à 22 h (17 exécutions par jour).
- Ne plus dépendre d'un crontab de la machine hôte ni de la commande *docker compose exec*.
- Garder un script **générique** : toute la logique métier est dans des fichiers SQL, organisés en un dossier par job.
- Pouvoir lancer à la demande tous les jobs, ou un seul, sans modifier le script.
- Tracer chaque exécution (lignes détectées, requête exécutée, détail avant et après) de façon lisible avec *kubectl logs*.

1.3 Avant et après

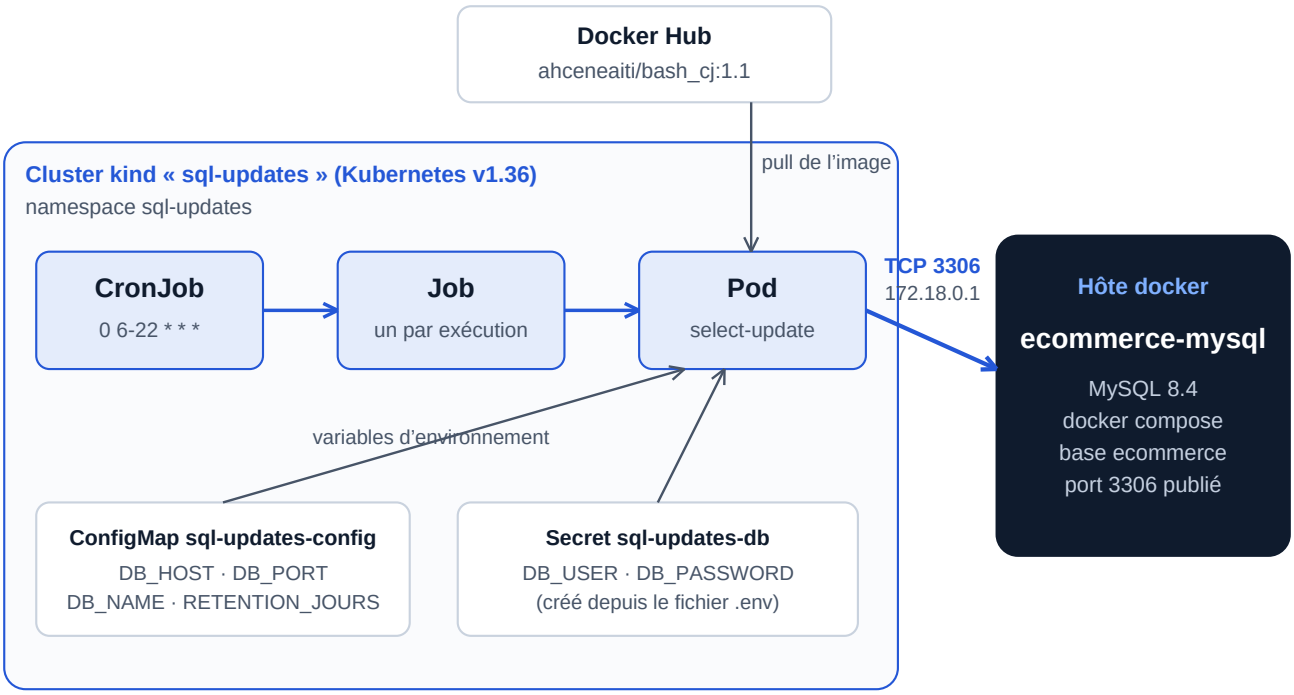
Aspect	Avant (cron hôte)	Après (CronJob Kubernetes)
Planification	crontab de l'utilisateur, lié à un chemin local	CronJob déclaratif : **0 6-22 * * ***, fuseau Europe/Paris
Exécution	`docker compose exec -T mysql select_update.sh`	Pod éphémère créé par un Job, un par échéance
Image	Script embarqué dans l'image MySQL	Image dédiée **ahceneaiti/bash_cj:1.1** (busybox + bash + client mysql)
Configuration	Variables du fichier .env et de docker-compose.yml	ConfigMap (réglages) et Secret (identifiants)
Logs	Fichier dans un dossier monté	Sortie standard, lue avec kubectl logs
Concurrence	Aucune protection	concurrencyPolicy: Forbid

1.4 Périmètre

MySQL reste volontairement **hors du cluster**, dans docker compose : le Dockerfile et le docker-compose.yml ne contiennent plus que le serveur (*init.sql* et *init_user.sh*). Le CronJob se connecte à ce serveur par le réseau. Les données de test (10 clients, 10 produits, 15 commandes, 10 livraisons) sont créées au premier démarrage du conteneur.

2. Architecture technique

2.1 Vue d'ensemble



Passerelle du réseau docker « kind » : adresse de l'hôte vue depuis les pods.

2.2 Composants

Composant	Rôle	Emplacement
CronJob sql-updates	Crée un Job à chaque échéance du planning	k8s/cronjob.yaml
Job / Pod	Exécute *select_update.sh* une fois, puis se termine (restartPolicy: Never)	créés par le CronJob
Image bash_cj:1.1	busybox + bash + client mysql + script + dossier sql/	Docker Hub : ahceneaiti/bash_cj:1.1
ConfigMap sql-updates-config	DB_HOST, DB_PORT, DB_NAME, RETENTION_JOURS	k8s/configmap.yaml
Secret sql-updates-db	DB_USER, DB_PASSWORD (non versionné)	créé avec kubectl depuis .env
emptyDir logs	Dossier /logs du pod pour le fichier de log (jetable)	volume du pod
ecommerce-mysql	Serveur MySQL 8.4 et données	docker compose, hôte

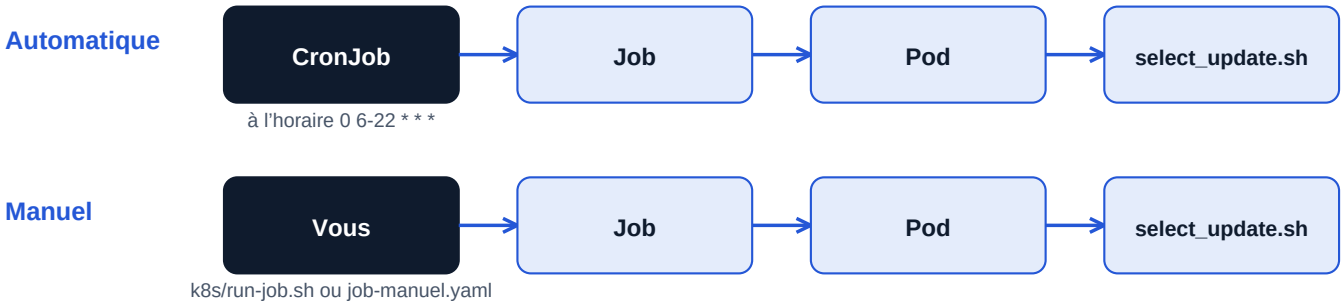
2.3 Déroulement d'une exécution

1. À l'heure prévue (minute 0, heures 6 à 22, fuseau Europe/Paris), le contrôleur de CronJob crée un **Job**. Si l'exécution précédente tourne encore, rien n'est créé (*Forbid*).
2. Le Job crée un **Pod**. Le nœud récupère l'image sur Docker Hub si elle n'est pas déjà présente (*imagePullPolicy: IfNotPresent*).
3. Kubernetes injecte les variables : tout le ConfigMap par *envFrom*, DB_USER et DB_PASSWORD depuis le Secret, LOG_DIR=/logs.

4. L'entrypoint *select_update.sh* démarre. Il lit ses arguments (aucun : tous les jobs), puis attend que MySQL réponde (30 essais espacés de 2 s).
5. Le script purge les anciens fichiers de log, puis traite chaque dossier de *sql/* : détection, liste d'id, détail, mise à jour, détail.
6. Tout est écrit sur la sortie standard et dans */logs/sql_updates_AAAA-MM-JJ.log*.
7. Le script se termine avec le code 0 (succès) ou 1 (job demandé introuvable). Le Job passe à *Complete* ou *Failed* ; les trois derniers Jobs réussis et échoués sont conservés.

Traitement automatique et traitement manuel

Dans les deux cas, le traitement est fait par un **Pod** créé par un **Job** ; seul change celui qui crée le Job.



	Automatique	Manuel
Déclencheur	L'horaire du CronJob (0 6-22 * * *)	Une commande : k8s/run-job.sh ou kubectl create -f k8s/job-manuel.yaml
Qui crée le Job	Le **CronJob** sql-updates, à chaque échéance	**Vous** : le Job est créé à la main
Qui crée le Pod	Le Job	Le Job
Nom du Job	sql-updates-<horodatage>	manuel-<horodatage> ou sql-updates-manuel-<suffixe>
Jobs SQL lancés	Tous (ou ceux de args)	Tous, ou un seul / une liste avec --job
Nettoyage	Historique : 3 réussis, 3 échoués	run-job.sh supprime le Job ; job-manuel.yaml expire après 1 h

Pour voir qui a créé chaque objet :

```
kubectl -n sql-updates get jobs -o custom-
    columns=JOB:.metadata.name,CREE_PAR:.metadata.ownerReferences[0].kind

JOB                       CREE_PAR
sql-updates-29903940      CronJob      <- automatique
sql-updates-manuel-jmdtg  <none>       <- manuel
```

2.4 Réseau

Le cluster est créé avec **kind** : ses nœuds sont des conteneurs docker, rattachés au réseau docker « kind ». Pour un pod, l'hôte docker est joignable par la **passerelle de ce réseau** (ici 172.18.0.1). Le port 3306, publié par docker compose, est donc accessible à cette adresse. La valeur se lit avec :

```
docker network inspect kind -f '{{range .IPAM.Config}}{{.Gateway}} {{end}}'
```

Avec minikube (driver docker), l'adresse à utiliser est *host.minikube.internal*. Le pare-feu de l'hôte doit laisser passer le port 3306.

2.5 Choix de conception

Décision	Raison
CronJob plutôt que cron de l'hôte	Planification déclarative, historique des Jobs, protection contre le chevauchement, aucune dépendance à un chemin local
MySQL hors du cluster	Conserver l'existant docker compose et ses données ; le CronJob ne dépend que d'une adresse et d'un port
Image busybox	Base minimale ; les deux seuls outils nécessaires (bash, client mysql) sont ajoutés par un stage de build
Script générique, SQL en fichiers	Ajouter ou modifier un job ne demande pas de toucher au script
Secret créé hors dépôt	Les identifiants ne sont jamais versionnés
Tag d'image explicite (1.1)	Évite qu'un nœud réutilise une ancienne image avec imagePullPolicy: IfNotPresent
Logs sur la sortie standard	Lecture par kubectl logs ; le fichier du pod est un emptyDir jetable

3. L'image ahceneaiti/bash_cj:1.1

L'image est basée sur **busybox**, comme demandé. Busybox ne contient ni bash ni client mysql ; le script utilise des tableaux bash et le client mysql est indispensable. Un build en deux stages règle ce point : le premier installe les outils, le second les copie sur busybox.

3.1 Dockerfile.cronjob

```
# Image du CronJob : base busybox + bash + client mysql + script + requêtes SQL.
# Le serveur MySQL reste dans son propre conteneur (docker compose), le job s'y connecte par le
    réseau.

# Stage 1 : installe bash et le client mysql dans un rootfs séparé (/out), sans apk ni cache
FROM alpine:3.20 AS deps
RUN apk add --no-cache --initdb --root /out \
        --keys-dir /etc/apk/keys --repositories-file /etc/apk/repositories \
        bash mariadb-client mariadb-connector-c \
    && rm -rf /out/lib/apk /out/var/cache /out/etc/apk /out/usr/share/man \
    && ln -sf mariadb /out/usr/bin/mysql

# Stage 2 : base busybox (musl) + dépendances copiées
FROM busybox:1.36-musl
COPY --from=deps /out/ /
COPY sql/ /usr/local/share/sql/
COPY select_update.sh /usr/local/bin/select_update.sh
RUN chmod +x /usr/local/bin/select_update.sh

ENTRYPOINT ["/usr/local/bin/select_update.sh"]
```

3.2 Explications

Instruction	Effet
`FROM alpine:3.20 AS deps`	Stage de build. Alpine fournit le gestionnaire de paquets apk et des paquets compilés pour musl, comme busybox:musl.
`apk add --initdb --root /out`	Installe les paquets dans un rootfs séparé /out, sans toucher au système du stage. Les options --keys-dir et --repositories-file réutilisent les clés et dépôts du stage.
`bash mariadb-client mariadb-connector-c`	Le shell, le client mysql (binaire *mariadb*) et les plugins d'authentification de la bibliothèque cliente.
`rm -rf /out/lib/apk ...`	Retire la base apk, les caches et les pages man : ils sont inutiles à l'exécution.
`ln -sf mariadb /out/usr/bin/mysql`	Le script appelle la commande *mysql* ; le lien la fait pointer vers le client mariadb.
`FROM busybox:1.36-musl`	Image finale : busybox avec la libc musl, compatible avec les binaires copiés depuis Alpine.
`COPY --from=deps /out/ /`	Recopie bash, le client et leurs bibliothèques (readline, ncurses, openssl, libstdc++...) dans l'image finale.
`COPY sql/ ...`	Place les jobs dans /usr/local/share/sql/ (variable SQL_DIR du script).
`COPY select_update.sh ...`	Installe le script dans /usr/local/bin/ et le rend exécutable.
`ENTRYPOINT [...]`	L'image lance directement le script. Les arguments du conteneur (args) sont passés au script.

3.3 Pourquoi mariadb-connector-c

MySQL 8.4 authentifie par défaut avec le plugin **caching_sha2_password**. Le client mariadb le charge depuis le dossier */usr/lib/mariadb/plugin*, fourni par le paquet *mariadb-connector-c*. Sans lui, la connexion échoue avec :

```
ERROR 1045 (28000): Plugin caching_sha2_password could not be loaded:
  Error loading shared library /usr/lib/mariadb/plugin/caching_sha2_password.so
```

Cette erreur a été rencontrée lors des premiers essais, puis corrigée en ajoutant le paquet au stage de build.

3.4 Contenu et taille

Chemin dans l'image	Contenu
/bin, /usr/bin (busybox)	Utilitaires : sh, awk, sed, find, paste, seq, date, tee, basename...
/usr/bin/bash, /usr/bin/mariadb, /usr/bin/mysql	Shell et client SQL (mysql est un lien vers mariadb)
/usr/lib/mariadb/plugin/	Plugins d'authentification, dont caching_sha2_password
/usr/local/bin/select_update.sh	Script, ENTRYPOINT de l'image
/usr/local/share/sql/	Dossiers de jobs : commandes, livraisons, produits

L'image locale pèse environ 122 Mo (121 881 956 octets), l'essentiel venant des bibliothèques du client MariaDB, d'OpenSSL et de libstdc++.

3.5 Construction, publication et versions

```
docker build -f Dockerfile.cronjob -t ahceneaiti/bash_cj:1.1 .
docker push ahceneaiti/bash_cj:1.1
```

Tag	Contenu	Digest
1.0	Première image : le script exécute tous les jobs (arguments positionnels)	sha256:3403049e1ac3...
1.1	Ajout de l'option --job, du code retour 1 pour un job inconnu et du code 2 pour une option invalide	sha256:f59026a4e466...

Règle : toute modification du script ou de *sql/* donne un **nouveau tag**, puis on met à jour *image:* dans k8s/cronjob.yaml. Avec *imagePullPolicy: IfNotPresent*, un tag réutilisé ne serait pas retiré par un nœud qui possède déjà l'ancienne image.

4. Manifestes Kubernetes

Les fichiers sont dans le dossier *k8s/* du dépôt.

Fichier	Rôle
namespace.yaml	Namespace **sql-updates**, qui regroupe tous les objets
configmap.yaml	Réglages non secrets
cronjob.yaml	CronJob planifié (tous les jobs par défaut)
job-manuel.yaml	Job unique à la demande, un job SQL via args
run-job.sh	Script de lancement à la demande à partir du CronJob
README.md	Procédure détaillée de déploiement

4.1 ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: sql-updates-config
  namespace: sql-updates
data:
  # Adresse du MySQL docker, vue depuis les pods :
  #   minikube (driver docker) : host.minikube.internal
  #   kind                     : IP de la passerelle du réseau docker "kind" (docker network
      inspect kind)
  DB_HOST: 172.18.0.1
  DB_PORT: "3306"
  DB_NAME: ecommerce
  RETENTION_JOURS: "7"
```

Clé	Valeur	Signification
DB_HOST	172.18.0.1	Adresse de l'hôte docker vue depuis les pods (passerelle du réseau kind)
DB_PORT	3306	Port MySQL publié par docker compose
DB_NAME	ecommerce	Base cible des requêtes
RETENTION_JOURS	7	Durée de conservation des fichiers de log dans /logs

4.2 Secret

Le Secret n'est pas dans le dépôt. Il est créé à partir du fichier *.env*, ignoré par git :

```
kubectl -n sql-updates create secret generic sql-updates-db --from-env-file=.env \
  --dry-run=client -o yaml | kubectl apply -f -
```

Le CronJob ne lit que les clés **DB_USER** et **DB_PASSWORD** (*secretKeyRef*) ; les autres clés du .env présentes dans le Secret sont ignorées. Pour changer le mot de passe, on modifie .env et on rejoue la commande : les prochains pods lisent la nouvelle valeur.

4.3 CronJob

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: sql-updates
  namespace: sql-updates
spec:
  # Toutes les heures pleines de 6h à 22h (comme le crontab d'origine)
  schedule: "0 6-22 * * *"
  timeZone: Europe/Paris
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 1
      activeDeadlineSeconds: 600
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: select-update
              image: ahceneaiti/bash_cj:1.1
              imagePullPolicy: IfNotPresent
              # Par défaut : tous les jobs de sql/. Pour ne lancer qu'un job planifié, décommenter
              # (--job est répétable ; valeurs : commandes, livraisons, produits)
              # args: ["--job", "commandes"]
              envFrom:
                - configMapRef:
                    name: sql-updates-config
              env:
                - name: DB_USER
                  valueFrom:
                    secretKeyRef:
                      name: sql-updates-db
                      key: DB_USER
                - name: DB_PASSWORD
                  valueFrom:
                    secretKeyRef:
                      name: sql-updates-db
                      key: DB_PASSWORD
                # Les logs vont aussi sur stdout (kubectl logs) ; le fichier est jetable
                - name: LOG_DIR
                  value: /logs
              volumeMounts:
                - name: logs
                  mountPath: /logs
              resources:
                requests: {cpu: 50m, memory: 64Mi}
                limits: {cpu: 200m, memory: 128Mi}
          volumes:
            - name: logs
              emptyDir: {}
```

Champ	Valeur	Effet
schedule	0 6-22 * * *	Minute 0 des heures 6 à 22, chaque jour : 6 h, 7 h, ... 22 h, soit 17 exécutions
timeZone	Europe/Paris	Le planning suit l'heure de Paris, y compris aux changements d'heure
concurrencyPolicy	Forbid	Si une exécution est encore en cours, la suivante n'est pas lancée
successfulJobsHistoryLimit	3	Nombre de Jobs réussis conservés (logs consultables)
failedJobsHistoryLimit	3	Nombre de Jobs échoués conservés
backoffLimit	1	Un nouvel essai en cas d'échec du pod
activeDeadlineSeconds	600	Arrêt forcé du Job au bout de 10 minutes
restartPolicy	Never	Le conteneur n'est pas redémarré dans le même pod
envFrom / env	ConfigMap + Secret	Injection des variables de configuration et des identifiants
volumes	emptyDir logs	Dossier /logs jetable, détruit avec le pod
resources	50m / 64Mi, 200m / 128Mi	Requests et limits CPU et mémoire, volontairement bas
args (commenté)	["--job", "commandes"]	À décommenter pour ne planifier que certains jobs

4.4 Job manuel (job-manuel.yaml)

Ce manifeste décrit un Job unique avec *args: ["--job", "commandes"]*, *backoffLimit: 0* et *ttlSecondsAfterFinished: 3600* (suppression automatique après une heure). Il utilise *generateName* : il se crée avec **kubectl create -f**, pas *apply*.

```
kubectl create -f k8s/job-manuel.yaml
kubectl -n sql-updates get jobs
```

4.5 Script run-job.sh

k8s/run-job.sh [dossier ...] lance des jobs à la demande en réutilisant le modèle du CronJob, donc la même image, les mêmes variables et le même secret :

1. Crée le manifeste d'un Job nommé *manuel-<timestamp>* avec *kubectl create job --from=cronjob/sql-updates --dry-run=client -o json*.
2. Si des dossiers sont donnés, un court script python3 ajoute *args* (une paire *--job dossier* par dossier) au conteneur.
3. Applique le Job, attend sa fin (180 s au plus), affiche les logs, puis supprime le Job.
4. En cas d'échec ou de délai dépassé, affiche quand même les logs et se termine avec le code 1.

5. Le script select_update.sh

5.1 Principe

Le script est **générique** : il ne contient aucune requête métier ni nom de fichier SQL. Il parcourt les sous-dossiers de *$SQL_DIR* (par défaut /usr/local/share/sql) dans l'ordre alphabétique. **Un dossier correspond à un job.** Ajouter un job revient à ajouter un dossier, sans modifier le script.

5.2 Arguments et code retour

```
select_update.sh [-j|--job dossier] [dossier ...]
```

Argument	Rôle
(aucun)	Tous les sous-dossiers de SQL_DIR, un par un
-j dossier, --job dossier	Ne lance que ce job ; option répétable
dossier	Forme positionnelle, équivalente à --job dossier
-h, --help	Affiche l'aide et se termine avec le code 0

Code retour	Situation
0	Exécution terminée, y compris quand aucun job ne détecte de ligne
1	Au moins un job demandé n'existe pas dans SQL_DIR : le Job Kubernetes passe en échec
2	Option inconnue, ou --job sans nom de dossier

Les arguments sont analysés au tout début, avant toute connexion à la base : une option invalide échoue immédiatement.

5.3 Variables d'environnement

Variable	Défaut	Dans le CronJob
DB_HOST	localhost	ConfigMap : 172.18.0.1
DB_PORT	3306	ConfigMap : 3306
DB_USER	root	Secret
DB_PASSWORD	root	Secret
DB_NAME	ecommerce	ConfigMap : ecommerce
LOG_DIR	/logs	env : /logs (emptyDir)
SQL_DIR	/usr/local/share/sql	valeur par défaut de l'image
RETENTION_JOURS	7	ConfigMap : 7

5.4 Traitement d’un job



Répété pour chaque dossier de sql/ ; si le SELECT ne renvoie rien, le job est terminé sans modification.

1. **Validation du dossier.** Il faut exactement un fichier de détection, exactement un *maj_*.sql* et au plus un *detail_*.sql*. Sinon le dossier est ignoré avec un message dans le log, sans erreur.
2. **Sélection.** Le fichier de détection est envoyé à mysql (options *-N -B* : sans en-tête, séparé par des tabulations). Si le résultat est vide, le log indique « Aucune ligne détectée » et le job se termine.
3. **Liste d’id.** La première colonne de chaque ligne est extraite avec awk, puis assemblée avec paste en *4,9,15*.
4. **Détail avant.** Si un *detail_*.sql* existe, il est exécuté avec la liste d’id et son résultat est loggé.
5. **Mise à jour.** Dans *maj_*.sql*, le marqueur **{{IDS}}** est remplacé par la liste (sed) ; la requête obtenue est loggée puis exécutée.
6. **Détail après.** Le même *detail_*.sql* est rejoué pour montrer l’état final des lignes.

5.5 Convention de nommage des fichiers

Fichier	Rôle	Obligatoire
autre *.sql (ex. commandes_abandonnees_france.sql)	Requête de détection ; la première colonne doit être l’id	oui, un seul
maj_*.sql	Requête de mise à jour, avec {{IDS}}	oui, un seul
detail_*.sql	Requête de détail, avec {{IDS}}, loggée avant et après	non, au plus un

Les lignes de commentaire SQL (--) des fichiers de mise à jour sont retirées avant l’affichage dans le log, mais pas de l’exécution.

5.6 Journalisation et rétention

- Chaque ligne est écrite sur la sortie standard et ajoutée au fichier *$LOG_DIR/sql_updates_AAAA-MM-JJ.log* (un fichier par jour, commun à tous les jobs).
- Chaque lancement commence par *=== Lancement à HH:MM (date) ===*, suivi d’une section par job, préfixée par *[dossier]*.
- Avant le traitement, les fichiers *sql_updates_*.log* plus vieux que RETENTION_JOURS jours (*find -mmin*) sont supprimés et listés dans le log.
- Dans Kubernetes, le fichier est dans un emptyDir : seule la sortie standard (kubectl logs) survit tant que le Job est conservé.

5.7 Attente de la base et gestion des erreurs

- Au démarrage, le script exécute *SELECT 1;* jusqu’à 30 fois, avec 2 s d’intervalle, pour laisser MySQL devenir joignable. Il continue ensuite sans autre vérification.

- Le script s'exécute avec *set -euo pipefail* : une commande mysql en échec arrête le script avec un code non nul, et le pod passe en *Error*.
- Les messages d'erreur de mysql sont masqués (*2>/dev/null*) pour ne pas afficher le mot de passe passé en argument. Conséquence : un échec de connexion se reconnaît à un log qui s'arrête juste après la ligne « Fichiers SQL », voir le dépannage (section 10).

6. Les jobs SQL fournis

Le dossier *sql/* contient trois jobs. Chacun a ses trois fichiers : détection, mise à jour, détail. Chaque *UPDATE* déclenche en plus le trigger *AFTER UPDATE* de la table, qui écrit l'état avant et après dans la table *historique*.

Job	Lignes détectées	Mise à jour	Fichiers
commandes	Commandes au statut ABONDONNE dont le client est en France	statut = 'OK'	commandes_abandonnees_france.sql maj_statut_ok.sql detail_commandes.sql
livraisons	Livraisons EN_ATTENTE des commandes de clients d'Italie	statut = 'EXPEDIE', date_expedition = date du jour	livraisons_en_attente_italie.sql maj_livraison_expediee.sql detail_livraisons.sql
produits	Produits dont le stock est inférieur à 10	stock = stock + 50	produits_stock_faible.sql maj_reappro_stock.sql detail_produits.sql

Le statut *ABONDONNE* est écrit tel quel (et non ABANDONNE), pour rester cohérent avec les données et les requêtes.

6.1 Job « commandes »

Détection, mise à jour, puis détail :

```
-- sql/commandes/commandes_abandonnees_france.sql
SELECT c.id, c.statut, cl.pays
FROM commande c
LEFT JOIN client cl ON cl.id = c.client_id
WHERE c.statut = 'ABONDONNE'
  AND cl.pays = 'France';

-- sql/commandes/maj_statut_ok.sql
UPDATE commande
SET statut = 'OK'
WHERE id IN ({{IDS}});

-- sql/commandes/detail_commandes.sql
SELECT c.id, c.statut, c.client_id, c.date, c.prix, c.produit_id, cl.pays
FROM commande c
LEFT JOIN client cl ON cl.id = c.client_id
WHERE c.id IN ({{IDS}});
```

6.2 Job « livraisons »

Détection, mise à jour, puis détail :

```
-- sql/livraisons/livraisons_en_attente_italie.sql
SELECT l.id, l.statut, cl.pays
FROM livraison l
JOIN commande c ON c.id = l.commande_id
LEFT JOIN client cl ON cl.id = c.client_id
WHERE l.statut = 'EN_ATTENTE'
  AND cl.pays = 'Italie';

-- sql/livraisons/maj_livraison_expediee.sql
```

```
UPDATE livraison
SET statut = 'EXPEDIE',
    date_expedition = CURDATE()
WHERE id IN ({{IDS}});

-- sql/livraisons/detail_livraisons.sql
SELECT l.id, l.commande_id, l.transporteur, l.statut, l.date_expedition, cl.nom, cl.pays
FROM livraison l
JOIN commande c ON c.id = l.commande_id
LEFT JOIN client cl ON cl.id = c.client_id
WHERE l.id IN ({{IDS}});
```

6.3 Job « produits »

Détection, mise à jour, puis détail :

```
-- sql/produits/produits_stock_faible.sql
SELECT id, stock
FROM produit
WHERE stock < 10;

-- sql/produits/maj_reappro_stock.sql
UPDATE produit
SET stock = stock + 50
WHERE id IN ({{IDS}});

-- sql/produits/detail_produits.sql
SELECT id, nom, categorie, prix, stock
FROM produit
WHERE id IN ({{IDS}});
```

6.4 Exemple de sortie

Extrait réel du log du job « commandes » exécuté dans le cluster :

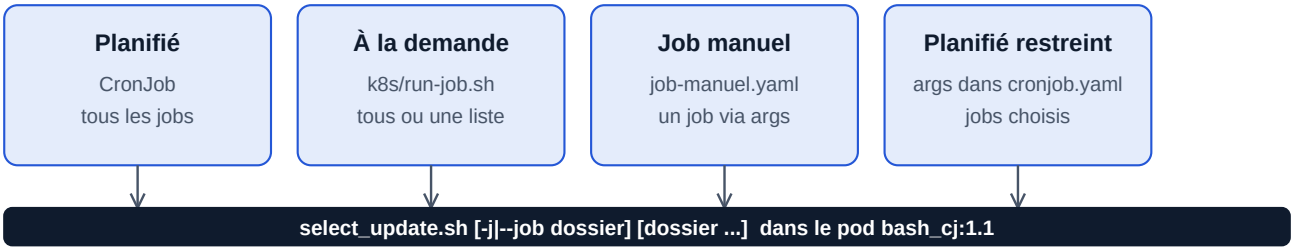
```
=== Lancement à 13:46 (2026-11-09) ===
[commandes] Fichiers SQL : select=commandes/commandes_abandonnees_france.sql | update=commandes/maj_statut_ok.sql |
    detail=commandes/detail_commandes.sql
--- Lignes détectées (commandes/commandes_abandonnees_france.sql) ---
4  ABONDONNE  France
9  ABONDONNE  France
15  ABONDONNE  France
--- Liste des id ---
4,9,15
--- Détail avant mise à jour (commandes/detail_commandes.sql) ---
4  ABONDONNE  4  2026-02-10  89.99  104  France
--- Requête exécutée (commandes/maj_statut_ok.sql) ---
UPDATE commande
SET statut = 'OK'
WHERE id IN (4,9,15);
--- Détail après mise à jour (commandes/detail_commandes.sql) ---
4  OK  4  2026-02-10  89.99  104  France
```

Quand les lignes ont déjà été traitées, le log contient seulement : *[commandes] Aucune ligne détectée par commandes/commandes_abandonnees_france.sql.*

6.5 Ajouter un job

1. Créer un dossier *sql/<domaine>/* avec une requête de détection (première colonne = id), un *maj_*.sql* utilisant {{IDS}}, et si besoin un *detail_*.sql*.
2. Reconstruire l'image avec un nouveau tag (par exemple 1.2) et la pousser sur Docker Hub.
3. Mettre à jour *image:* dans k8s/cronjob.yaml (et job-manuel.yaml), puis *kubectl apply -f k8s/cronjob.yaml*.
4. Le job est pris en compte automatiquement, sans modifier le script ; il est lançable avec *k8s/run-job.sh <domaine>*.

7. Modes de lancement



7.1 Lancement planifié

Le CronJob crée un Job à chaque heure pleine de 6 h à 22 h. Sans argument, le script traite tous les dossiers de *sql/*. Vérification de l'état :

```
kubectl -n sql-updates get cronjob
NAME          SCHEDULE       TIMEZONE       SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sql-updates   0 6-22 * * *   Europe/Paris   False     0        92s             14m
```

7.2 Lancement à la demande avec run-job.sh

```
k8s/run-job.sh                    # tous les jobs
k8s/run-job.sh livraisons         # un seul job
k8s/run-job.sh commandes produits  # plusieurs jobs
```

Le script crée un Job à partir du CronJob, attend sa fin, affiche les logs, puis supprime le Job. Si le CronJob contient lui-même des *args*, un lancement sans dossier les conserve ; avec un ou plusieurs dossiers, ils sont remplacés pour ce lancement.

7.3 Job manuel (manifeste)

On édite la ligne *args* de *k8s/job-manuel.yaml*, puis :

```
kubectl create -f k8s/job-manuel.yaml
kubectl -n sql-updates wait --for=condition=complete job -l job-name --timeout=120s
```

Le Job porte un nom généré (*sql-updates-manuel-xxxxx*) et se supprime seul une heure après sa fin.

7.4 Un seul job planifié : args dans cronjob.yaml

Par défaut, *cronjob.yaml* contient une ligne *args* en commentaire, sous *imagePullPolicy* :

```
# args: ["--job", "commandes"]
```

1. Décommenter la ligne et choisir le job (commandes, livraisons ou produits). Pour plusieurs jobs : *["--job", "commandes", "--job", "produits"]*.
2. Appliquer : *kubectl apply -f k8s/cronjob.yaml*.
3. Vérifier : *kubectl -n sql-updates get cronjob sql-updates -o jsonpath='{.spec.jobTemplate.spec.template.spec.containers[0].args}'*.
4. Pour revenir à tous les jobs, recommenter la ligne et refaire *kubectl apply*.

7.5 Comparatif

Mode	Déclencheur	Jobs lancés	Nettoyage du Job
Planifié	Horaire du CronJob	Tous (ou ceux de args)	Historique : 3 réussis et 3 échoués
run-job.sh	Commande manuelle	Tous ou liste donnée	Supprimé par le script
job-manuel.yaml	kubectl create -f	Ceux de args dans le manifeste	Auto après 1 h (TTL)
args du CronJob	Horaire + kubectl apply	Ceux de args	Historique : 3 réussis et 3 échoués

Un nom de job inconnu fait échouer le Job (code retour 1) : le log indique « Dossier introuvable dans /usr/local/share/sql ».

8. Mise en route pas à pas

Toutes les commandes sont lancées depuis la racine du projet. Prérequis : docker, docker compose v2, kubectl, kind, python3 (pour run-job.sh) et un accès à Docker Hub (docker login) pour publier l'image.

Étape 1. Préparer la configuration

```
cp .env.example .env     # puis adapter DB_USER, DB_PASSWORD, DB_NAME
```

Étape 2. Lancer MySQL

```
docker compose up -d --build
docker compose ps                # attendre l'état « healthy »
```

Étape 3. Construire et publier l'image

```
docker build -f Dockerfile.cronjob -t ahceneaiti/bash_cj:1.1 .
docker push ahceneaiti/bash_cj:1.1
```

Étape 4. Créer le cluster

```
kind create cluster --name sql-updates
kubectl config use-context kind-sql-updates
kubectl get nodes                # le nœud doit être Ready
```

Étape 5. Renseigner DB_HOST

```
docker network inspect kind -f '{{range .IPAM.Config}}{{.Gateway}} {{end}}'
```

Reporter l'adresse IPv4 (par exemple 172.18.0.1) dans *k8s/configmap.yaml*, clé *DB_HOST*.

Étape 6. Déployer

```
kubectl apply -f k8s/namespace.yaml -f k8s/configmap.yaml
kubectl -n sql-updates create secret generic sql-updates-db --from-env-file=.env \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -f k8s/cronjob.yaml
kubectl -n sql-updates get cronjob
```

Étape 7. Tester sans attendre l'horaire

```
k8s/run-job.sh                   # lance tous les jobs et affiche les logs
k8s/run-job.sh livraisons        # un seul job
```

Étape 8. Réinitialiser les données de test

Les données ne sont pas conservées dans un volume : pour retrouver des lignes à traiter, recréer le conteneur MySQL.

```
docker compose down && docker compose up -d
```

Suppression complète

```
kubectl delete namespace sql-updates     # CronJob, ConfigMap, Secret
kind delete cluster --name sql-updates
docker compose down
```

9. Exploitation

9.1 Commandes courantes

Action	Commande
Voir les exécutions	`kubectl -n sql-updates get jobs,pods`
Lire les logs d'une exécution	`kubectl -n sql-updates logs job/<nom>`
Voir l'état du CronJob	`kubectl -n sql-updates get cronjob sql-updates`
Suspendre	`kubectl -n sql-updates patch cronjob sql-updates -p '{"spec":{"suspend":true}}'`
Reprendre	`kubectl -n sql-updates patch cronjob sql-updates -p '{"spec":{"suspend":false}}'`
Supprimer un Job	`kubectl -n sql-updates delete job <nom>`

9.2 Logs et rétention

Les logs sont sur la sortie standard. Le fichier *sql_updates_*.log* du pod est dans un emptyDir détruit avec le pod : la rétention réelle est celle des Jobs conservés par le CronJob (*successfulJobsHistoryLimit* et *failedJobsHistoryLimit*, soit 3 chacun). La variable RETENTION_JOURS ne prend son sens que si /logs devient un volume persistant.

9.3 Publier une nouvelle version

1. Modifier *select_update.sh* ou *sql/*.
2. Construire et pousser une nouvelle image : *docker build -f Dockerfile.cronjob -t ahceneaiti/bash_cj:1.2 .* puis *docker push ahceneaiti/bash_cj:1.2*.
3. Changer le tag dans *k8s/cronjob.yaml* (et *job-manuel.yaml*).
4. Appliquer : *kubectl apply -f k8s/cronjob.yaml*. Les prochains Jobs utilisent la nouvelle image.
5. Tester avec *k8s/run-job.sh*, puis commiter.

9.4 Modifier la configuration

- ConfigMap : modifier *k8s/configmap.yaml* puis *kubectl apply -f k8s/configmap.yaml* ; pris en compte au prochain lancement.
- Secret : modifier *.env* et rejouer la commande de création du Secret.
- Planning : modifier *schedule* dans *cronjob.yaml* puis appliquer.

10. Dépannage

Symptôme	Cause probable	Action
Pod en **Error**, log arrêté après « Fichiers SQL : ... »	Connexion MySQL refusée ; les erreurs mysql sont masquées	Vérifier DB_HOST, le port 3306, le pare-feu de l'hôte et les identifiants du Secret
Log « Plugin caching_sha2_password could not be loaded »	Image sans mariadb-connector-c	Reconstruire l'image avec Dockerfile.cronjob à jour
ImagePullBackOff	Image absente de Docker Hub ou tag incorrect	Vérifier le tag dans cronjob.yaml et que le push a réussi
« Dossier introuvable dans /usr/local/share/sql »	Nom de job inconnu dans --job ou run-job.sh	Utiliser un sous-dossier de sql/ : commandes, livraisons, produits
« Aucune ligne détectée »	Les lignes ont déjà été mises à jour	Normal ; réinitialiser les données de test (étape 8) pour rejouer
Aucune exécution à l'heure prévue	CronJob suspendu, ou horaire mal interprété	Vérifier SUSPEND et le fuseau Europe/Paris
Dossier ignoré, message dans le log	Il n'y a pas exactement 1 détection, 1 maj_*.sql et au plus 1 detail_*.sql	Corriger le contenu du dossier et republier l'image
Option inconnue, code 2	Argument invalide passé à select_update.sh	Utiliser -j ou --job suivi d'un nom de dossier

Pour ouvrir un shell dans un pod avec les mêmes outils :

```
kubectl -n sql-updates run -it --rm dbg --image=ahceneaiti/bash_cj:1.1 --restart=Never --command
    -- sh
```

11. Sécurité et limites

11.1 Sécurité

- Les identifiants ne sont jamais dans le dépôt : *.env* est ignoré par git et le Secret est créé hors dépôt. Seul *.env.example* est versionné.
- Un Secret Kubernetes est encodé en base64, pas chiffré, tant que le chiffrement au repos du cluster n'est pas activé.
- Les valeurs par défaut (utilisateur *root*, mot de passe *root*) ne conviennent que pour un test local. En production, utiliser un utilisateur dédié (*init_user.sh* en crée un si DB_USER n'est pas root) et un mot de passe fort.
- Le mot de passe est passé en argument à mysql ; les erreurs mysql sont masquées pour éviter de l'afficher dans les logs.
- Les requêtes SQL sont des fichiers du dépôt, relus avant publication : le marqueur {{IDS}} n'est remplacé que par des id issus de la base.

11.2 Limites connues

- **Dépendance à l'hôte.** MySQL tourne dans docker compose : si l'hôte ou le conteneur est arrêté, les exécutions échouent.
- **Adresse en dur.** DB_HOST est la passerelle du réseau kind ; elle peut changer si le réseau docker est recréé et doit alors être mise à jour.

- **Pas d'atomicité entre SELECT et UPDATE.** La liste d'id est figée après le SELECT, mais une ligne modifiée entre-temps par une autre source serait quand même mise à jour.
- **Erreurs masquées.** La cause exacte d'un échec mysql n'apparaît pas dans le log ; elle se déduit de l'endroit où le log s'arrête.
- **Logs non persistants.** Le fichier de log disparaît avec le pod ; seuls les Jobs conservés (3 + 3) gardent leur sortie standard.
- **Mise à jour de l'image manuelle.** Chaque modification demande un build, un push et un changement de tag.
- **Fuseau et changement d'heure.** Le planning suit Europe/Paris (champ timeZone), ce qui requiert une version de Kubernetes qui le gère (cluster v1.36.1 ici).

11.3 Évolutions possibles

Ces pistes sont des suggestions, elles ne sont pas réalisées :

- Déployer MySQL dans le cluster (StatefulSet et volume persistant) pour supprimer la dépendance à l'hôte.
- Externaliser les secrets (Sealed Secrets, gestionnaire de secrets) et activer le chiffrement au repos.
- Conserver les logs sur un volume persistant ou les envoyer vers un outil de collecte.
- Automatiser le build, le push et le changement de tag dans une chaîne d'intégration continue.
- Afficher l'erreur mysql (hors mot de passe) dans le log pour accélérer le diagnostic.

Annexe A. Arborescence du dépôt

```
bash_sql/
  Dockerfile              image du serveur MySQL (init.sql, init_user.sh)
  docker-compose.yml      service mysql (port 3306, healthcheck)
  Dockerfile.cronjob      image du CronJob (busybox + bash + client mysql)
  select_update.sh        script générique, ENTRYPOINT de l'image du CronJob
  init.sql                tables, triggers et jeu de données
  init_user.sh            création de DB_USER s'il n'est pas root
  .env.example            modèle de configuration (DB_USER, DB_PASSWORD, DB_NAME)
  sql/
    commandes/            commandes_abandonnees_france.sql, maj_statut_ok.sql,
        detail_commandes.sql
    livraisons/           livraisons_en_attente_italie.sql, maj_livraison_expediee.sql,
        detail_livraisons.sql
    produits/             produits_stock_faible.sql, maj_reappro_stock.sql, detail_produits.sql
  k8s/
    namespace.yaml        namespace sql-updates
    configmap.yaml        DB_HOST, DB_PORT, DB_NAME, RETENTION_JOURS
    cronjob.yaml          CronJob planifié
    job-manuel.yaml       Job unique à la demande
    run-job.sh            lancement à la demande depuis le CronJob
    README.md             procédure Kubernetes détaillée
  docs/                   présentations
  README.md               documentation générale
```

Annexe B. Historique des commits liés au CronJob

Commit	Message
`c932a85`	docs: documenter l'argument --job optionnel du CronJob
`cab1ead`	refactor: retirer le script et sql/ de l'image MySQL et du compose
`bff17ec`	feat: argument --job optionnel (commenté) dans le CronJob
`df0a7a3`	docs: démarrage rapide et dépannage Kubernetes
`96be446`	feat: argument optionnel --job pour lancer un seul job SQL
`fed75d7`	docs: lancer le job depuis le CronJob et ajouter k8s/README.md
`f3a3c15`	feat: déploiement Kubernetes en CronJob avec image busybox
`edc7488`	docs: ajouter la présentation PDF de l'automatisation SQL
`5fe3bb6`	feat: configuration via fichier .env (utilisateur, mot de passe, base, logs)
`12a10a2`	refactor: parcourir le dossier sql/ au lieu de noms de fichiers en dur
`ab54db7`	refactor: renommer le fichier de log maj_commandes_ en sql_updates_
`33691bb`	feat: table historique alimentée par triggers AFTER UPDATE

Annexe C. Script select_update.sh (intégral)

Les lignes de plus de 118 caractères sont coupées sur plusieurs lignes dans cette annexe ; le fichier d'origine ne contient pas ces coupures.

```
#!/usr/bin/env bash
# Script générique : parcourt les dossiers de $SQL_DIR (un dossier = un job). Pour chaque dossier,
# il se connecte à la base, exécute la requête SELECT, construit la liste d'id (1re colonne du résultat),
# exécute la requête UPDATE sur cette liste et journalise le tout. Toute la logique métier est dans les .sql.
#
# Usage : select_update.sh [-j|--job dossier] [dossier ...]
#   sans argument      : tous les sous-dossiers de $SQL_DIR, dans l'ordre alphabétique
#   -j, --job dossier  : ne lance que ce job (option répétable) ; équivalent à passer le dossier seul
#   dossier            : nom d'un sous-dossier de $SQL_DIR (ex. commandes) pour ne lancer que ce job
#   -h, --help         : affiche cette aide
# Code retour : 0 si tout s'est bien passé, 1 si un job demandé n'existe pas, 2 en cas d'option invalide.
# Convention de nommage dans un dossier :
#   maj_*.sql     : requête de mise à jour (obligatoire), le marqueur {{IDS}} est remplacé par la liste d'id
#   detail_*.sql  : requête de détail (optionnelle), même marqueur, loggée avant/après la mise à jour
#   autre *.sql   : requête de détection (obligatoire), 1re colonne = id
set -euo pipefail

usage() {
    echo "Usage : $(basename "$0") [-j|--job dossier] [dossier ...]"
    echo "  sans argument : tous les jobs du dossier SQL_DIR"
    echo "  -j, --job     : ne lance que ce job (répétable)"
}

# Arguments : jobs demandés (option -j/--job ou dossier seul) ; vide = tous les jobs
jobs=()
while [ "$#" -gt 0 ]; do
    case "$1" in
        -j|--job)
            if [ "$#" -lt 2 ] || [ -z "$2" ]; then
                echo "Option $1 : nom de dossier manquant." >&2
                usage >&2
                exit 2
            fi
            jobs+=("$2")
            shift 2
            ;;
        -h|--help) usage; exit 0 ;;
        -*) echo "Option inconnue : $1" >&2; usage >&2; exit 2 ;;
        *) jobs+=("$1"); shift ;;
    esac
done

DB_HOST="${DB_HOST:-localhost}"
DB_PORT="${DB_PORT:-3306}"
DB_USER="${DB_USER:-root}"
DB_PASSWORD="${DB_PASSWORD:-root}"
DB_NAME="${DB_NAME:-ecommerce}"
LOG_DIR="${LOG_DIR:-/logs}"
RETENTION_JOURS="${RETENTION_JOURS:-7}"
SQL_DIR="${SQL_DIR:-/usr/local/share/sql}"

mkdir -p "$LOG_DIR"
# Un seul fichier de log par jour pour tous les jobs ; chaque lancement y ajoute un bloc horodaté (heure:minute)
LOG_FILE="$LOG_DIR/sql_updates_$(date '+%Y-%m-%d').log"
HEURE=$(date '+%H:%M')
ids=""

run_sql() {
    mysql -h "$DB_HOST" -P "$DB_PORT" -u "$DB_USER" -p"$DB_PASSWORD" "$DB_NAME" -N -B -e "$1" 2>/dev/null
}

run_sql_file() {
    mysql -h "$DB_HOST" -P "$DB_PORT" -u "$DB_USER" -p"$DB_PASSWORD" "$DB_NAME" -N -B < "$1" 2>/dev/null
}

# Lit un fichier .sql et remplace {{IDS}} par la liste d'id (valeurs issues de la base)
render_sql() {
    sed "s/{{IDS}}/$ids/g" "$1"
}

log() { echo "$*" | tee -a "$LOG_FILE"; }

# Liste les fichiers .sql d'un dossier selon le préfixe : $1 = dossier, $2 = maj | detail | select
list_sql() {
    local f base
    for f in "$1"/*.sql; do
        [ -e "$f" ] || continue
        base=$(basename "$f")
```

```
        case "$2:$base" in
            maj:maj_*|detail:detail_*) echo "$f" ;;
            select:maj_*|select:detail_*) ;;
            select:*) echo "$f" ;;
        esac
    done
}

# Exécute un job : $1 = nom du dossier dans $SQL_DIR
run_job() {
    local job="$1" dir="$SQL_DIR/$1" select_files update_files detail_files
    local select_f update_f detail_f="" rows update_sql

    select_files=$(list_sql "$dir" select)
    update_files=$(list_sql "$dir" maj)
    detail_files=$(list_sql "$dir" detail)

    if [ "$(echo -n "$select_files" | grep -c .)" -ne 1 ] || [ "$(echo -n "$update_files" | grep -c .)" -ne 1 ] \
        || [ "$(echo -n "$detail_files" | grep -c .)" -gt 1 ]; then
        log "[${job}] Dossier ignoré : il faut exactement 1 select, 1 maj_*.sql et au plus 1 detail_*.sql."
        log ""
        return 0
    fi
    select_f="$select_files"
    update_f="$update_files"
    detail_f="$detail_files"

    log "[${job}] Fichiers SQL : select=${select_f#"$SQL_DIR"/} | update=${update_f#"$SQL_DIR"/} | detail=$([ -n
        "$detail_f" ] && echo "${detail_f#"$SQL_DIR"/}" || echo aucun)"

    # 1. Sélection
    ids=""
    rows=$(run_sql_file "$select_f")
    if [ -z "$rows" ]; then
        log "[${job}] Aucune ligne détectée par ${select_f#"$SQL_DIR"/}."
        log ""
        return 0
    fi

    # 2. Construction de la liste d'id : 4,7,9,... (1re colonne)
    ids=$(echo "$rows" | awk '{print $1}' | paste -sd, -)

    log "--- Lignes détectées (${select_f#"$SQL_DIR"/}) ---"
    log "$rows"
    log "--- Liste des id ---"
    log "$ids"
    if [ -n "$detail_f" ]; then
        log "--- Détail avant mise à jour (${detail_f#"$SQL_DIR"/}) ---"
        log "$(run_sql "$(render_sql "$detail_f")")"
    fi

    # 3. Mise à jour
    update_sql=$(render_sql "$update_f")
    log "--- Requête exécutée (${update_f#"$SQL_DIR"/}) ---"
    log "$(echo "$update_sql" | grep -v '^--')"
    run_sql "$update_sql"

    # 4. Lignes mises à jour
    if [ -n "$detail_f" ]; then
        log "--- Détail après mise à jour (${detail_f#"$SQL_DIR"/}) ---"
        log "$(run_sql "$(render_sql "$detail_f")")"
    fi
    log ""
}

# Attente que la base accepte les connexions
for _ in $(seq 1 30); do
    run_sql "SELECT 1;" >/dev/null 2>&1 && break
    sleep 2
done

log "=== Lancement à ${HEURE} ($(date '+%F')) ==="

# Rétention : suppression des logs plus vieux que RETENTION_JOURS jours
purges=$(find "$LOG_DIR" -maxdepth 1 -type f -name 'sql_updates_*.log' \
    -mmin +$((RETENTION_JOURS * 1440)) -print -delete)
if [ -n "$purges" ]; then
    log "--- Logs supprimés (rétention ${RETENTION_JOURS} jours) ---"
    log "$purges"
fi

# Jobs : dossiers demandés, sinon tous les sous-dossiers de $SQL_DIR
if [ "${#jobs[@]}" -eq 0 ]; then
    for d in "$SQL_DIR"/*/; do
        [ -d "$d" ] && jobs+=("$(basename "$d")")
```

```
    done
fi

if [ "${#jobs[@]}" -eq 0 ]; then
    log "Aucun dossier de requêtes dans ${SQL_DIR}."
    log ""
    exit 0
fi

erreurs=0
for job in "${jobs[@]}"; do
    if [ ! -d "$SQL_DIR/$job" ]; then
        log "[${job}] Dossier introuvable dans ${SQL_DIR}."
        log ""
        erreurs=1
        continue
    fi
    run_job "$job"
done
exit "$erreurs"
```

Annexe D. Manifeste job-manuel.yaml

```
# Lancement manuel d'un seul job SQL (hors planification du CronJob).
# Le dossier de sql/ est passé en argument : remplacer "commandes" par le job voulu
# (commandes, livraisons, produits) ou utiliser le script k8s/run-job.sh.
# Création : kubectl create -f k8s/job-manuel.yaml   (generateName : pas de "apply")
apiVersion: batch/v1
kind: Job
metadata:
  generateName: sql-updates-manuel-
  namespace: sql-updates
spec:
  backoffLimit: 0
  activeDeadlineSeconds: 600
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: select-update
          image: ahceneaiti/bash_cj:1.1
          imagePullPolicy: IfNotPresent
          args: ["--job", "commandes"]
          envFrom:
            - configMapRef:
                name: sql-updates-config
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: sql-updates-db
                  key: DB_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: sql-updates-db
                  key: DB_PASSWORD
            - name: LOG_DIR
              value: /logs
          volumeMounts:
            - name: logs
              mountPath: /logs
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits: {cpu: 200m, memory: 128Mi}
      volumes:
        - name: logs
          emptyDir: {}
```

Annexe E. Script k8s/run-job.sh

```
#!/usr/bin/env bash
# Lance un job SQL à la demande depuis le CronJob, puis affiche ses logs.
# Usage : k8s/run-job.sh [dossier ...]   (sans argument : tous les jobs)
# Exemple : k8s/run-job.sh livraisons
set -euo pipefail

NS="${NS:-sql-updates}"
CRONJOB="${CRONJOB:-sql-updates}"
name="manuel-$(date +%s)"

args=()
for j in "$@"; do args+=(--job "$j"); done

# Copie le modèle du CronJob (image, env, secret) et ajoute les arguments au conteneur
kubectl -n "$NS" create job "$name" --from="cronjob/$CRONJOB" --dry-run=client -o json \
    | ARGS="$(printf '%s\n' "${args[@]+"${args[@]}"}")" python3 -c '
import json, os, sys
j = json.load(sys.stdin)
a = [x for x in os.environ["ARGS"].split("\n") if x]
if a:
    j["spec"]["template"]["spec"]["containers"][0]["args"] = a
print(json.dumps(j))' \
    | kubectl apply -f - >/dev/null

kubectl -n "$NS" wait --for=condition=complete "job/$name" --timeout=180s \
    || { kubectl -n "$NS" logs "job/$name" || true; exit 1; }
kubectl -n "$NS" logs "job/$name"
kubectl -n "$NS" delete "job/$name" >/dev/null
```